

Buffer Overflow Attacks

2. Running ./main 6 returns:

Target Function Ptr: 0x4011dc

3.

Running ./main 8 gives the same output, but when an offset of 50 is used we get a segmentation fault error.

Checking more carefully, the offset threshold for which we get a 'segmentation fault (core dumped)' is at 14.

At an offset of 20, the error changes to 'bus error (core dumped)', however this only occurs for offsets 20 and 21, and all other offsets above 14 give the segmentation error, not the bus error.

At offset 24, we see that the program returns, 'Success! Malicious Function Called', suggesting that the buffer address was overwritten with a pointer to the malicious function for this offset.

The target function pointer for all runs of the function with all offsets is always 0x4011dc

4.

for an 8 byte buffer, the offset required to overwrite the return address is:

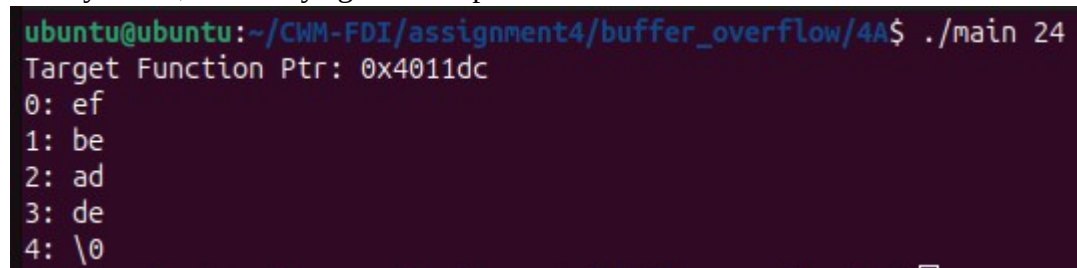
8 bytes (buffer)
+ 8 bytes (saved RBP)
+ 8 bytes (to reach the return address)
= 24 bytes offset to overwrite the return address

5. any offset over 12 causes a segmentation fault (except for 24 of course). This is because small offsets are within the current stack frame (buffer + saved RBP + part of stack address), so all of this memory is already allocated and valid.

For large offsets, overflow is beyond the current frame, so we cross into unallocated stack space. Modern systems use guard pages (deliberately unallocated stack space) that if overflowed into will cause an immediate segmentation fault.

6.

for any offset, we always get the output:



```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 24
Target Function Ptr: 0x4011dc
0: ef
1: be
2: ad
3: de
4: \0
```

We can see that the order of the bytes is reversed compared to how 0xdeadbeef is written in hex.

7. strcpy(dest, src) copies bytes until it hits a null terminator, with zero length checking. If src is longer than dest's allocation, strcpy will just blindly overwrite adjacent memory. The function has no way to know dest's size

A better alternative is `strncpy(dest, src, sizeofbuffer)`, where you set a buffer area to copy to, however you must manually null-terminate the area, if `src >= n` bytes, which can be slow

Buffer Overflow Attacks – Mounting Your First Attack

8. setup

9. reading

10.

for username: I get 3 attempts each returning invalid username and then I am exited – no seg fault
for password: I get the same ‘too many failed attempts’ error and am again exited from the program

for ‘enter your name’, works perfectly

for ‘enter your department’, works perfectly

for ‘enter system command’, works perfectly

for ‘enter response data’, we get out segmentation fault → this is the vulnerable function.

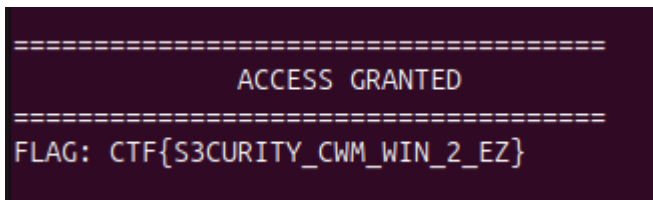
11. so, the vulnerable function is enter response data.

12. exploit.py found that an offset of 264 bytes caused a segmentation fault.

13. exploit.py says that the addresses are:

win: 0x401216

main: 0x401679



used target = win address, and offset of 264, found earlier.

Timing attacks

14. – 17. setup tasks

18.

the reason a correct leading character takes longer than an incorrect one is that a correct one calls the sleep function, which pauses the entire loop for 0.002 seconds.

19. as if correct characters induce a longer response, we can figure out exactly which characters are correct by trying many and looking at each one’s response time to find the whole password

20.

0 – 0.0

1 – 0.0

2 – 0.0

3 – 0.0
4 – 0.0
5 – 2.2
6 – 0.0
7 – 0.0
8 – 0.0
9 – 0.0

21. the guess of the number 5 took noticeably longer than the others, suggesting that the first digit of the password is a 5

22.

trying with the second character, all attempts took the same amount of time as the first correct character, except the attempt 51, which took 4.3 seconds. This suggests that the first two digits of the password are 5 and 1

23. – 24. setup

25.

The output prints the html of the successfully logged in page – ie. the attack cracked the password and logged us in.

```
[+] prefix so far: '519265'

[*] Password cracked: '519265'
<!DOCTYPE html>
<html>
<head><title>Login</title>
<style>
  body { font-family: monospace; max-width: 480px; margin: 60px auto; }
  .result { margin-top: 16px; padding: 12px; border: 1px solid #ccc; }
  .error { color: #c00; }
  .ok { color: #080; }
  .time { font-size: 1.4em; font-weight: bold; }
</style>
</head>
<body>
  <h2>Login</h2>

  <div class="result ok">
    <p>Access granted. Flag: <strong>CWM{t1m1ng_1s_3v3ryth1ng}</strong></p>
    <p>Check took: <span class="time">12.8 ms</span></p>
  </div>

  <form method="post" style="margin-top:24px">
    <input type="hidden" name="csrfmiddlewaretoken" value="78dYjk86twrYnUsI9vPKhalsPMKbuL0nScIZpiedlDgVt0bk8EislxJqE1YXlpk9">
    <label>Username<br><input type="text" name="username" value="admin"></label><br><br>
    <label>Password<br><input type="text" name="password" autocomplete="off"></label><br><br>
    <button type="submit">Try</button>
  </form>
</body>
</html>
```

as we can see, the access granted flag is: **CWM{t1m1ng_1s_3v3ryth1ng}**

26.

```
def safe_check(username: str, password: str) -> bool:
    """
    stub for safe password check
    To be completed by you!
    """
    if username != SECRET_USERNAME:
        return False

    result = True

    for i, ch in enumerate(password):
        # i == index of the current character
        # ch == current character

        # code below is vulnerable. Fix it!
        # then replace _vulnerable_check with safe_check in login_view() below
        if i >= len(SECRET_PASSWORD) or ch != SECRET_PASSWORD[i]:
            return False

    if len(password) == len(SECRET_PASSWORD):
        return True
    else:
        return False
```

To make the login safe, we just have to remove the `time.sleep` and rewrite the code so it makes logical sense.

Once we do that, the `attack.py` script does not pick up on significant time differences as there are no longer any, and instead responds to small fluctuations of noise, ie. it fills in random values for each number indefinitely until the program is stopped.

27.

you would need to make sure that the often long API response times are not something that can be used to determine information about the system. If every successful character was an api call, that could introduce the same sort of delay as seen with `time.sleep()`, that could then be measured and extracted.

You could try to fix this by adding a delay to both sides depending on the API response time, but if that response time fluctuates that could be an issue. In that case you could then maybe add synthetic random time delay fluctuations to mask the API's effect.